

DATA STRUCTURE — UNIT 3

Complete Exam-Preparation Package: Stack & Queue

Syllabus covered: 3.1 Introduction to Stack, 3.2 Applications of Stack, 3.3 Introduction to Queue, 3.4 Types of Queue.

Goal: Prepare for a 30/30 test using definitions, diagrams, algorithms, conversions, evaluation problems, differences, programs, MCQs, viva questions and a model paper.

How to use: First learn the concepts, then solve the worked examples without looking, then attempt the model paper under exam conditions.

3.1 INTRODUCTION TO STACK

Definition: A stack is a linear data structure in which insertion and deletion are performed only at one end called TOP. It follows LIFO (Last In First Out).

Real-life example: A pile of plates. The last plate placed on top is the first plate removed.

Basic operations: Push = insert, Pop = delete, Peek/Top = view top element without deleting it, isEmpty = check empty, isFull = check full.

Diagram:

```
TOP → [ 30 ]  
[ 20 ]  
[ 10 ]
```

ADT: An Abstract Data Type specifies what data and operations are available without requiring a particular implementation. A stack ADT can be implemented using an array or linked list.

Overflow: Trying to push into a full stack. **Underflow:** Trying to pop from an empty stack.

Array representation

For an array stack of size MAX, TOP is commonly initialized to -1. The top element is STACK[TOP].

Push algorithm

1. If $TOP == MAX-1$, report Stack Overflow.
2. Otherwise $TOP = TOP + 1$.
3. $STACK[TOP] = ITEM$.
4. Stop.

Pop algorithm

1. If $TOP == -1$, report Stack Underflow.
2. Otherwise $ITEM = STACK[TOP]$.
3. $TOP = TOP - 1$.
4. Return ITEM.

Peek algorithm: If $TOP == -1$, stack is empty; otherwise return $STACK[TOP]$.

Time complexity: Push, Pop and Peek are $O(1)$.

Linked-list representation

Each node contains DATA and NEXT. TOP points to the first node. Push and pop are normally performed at the beginning of the linked list.

```
TOP → [30 | next] → [20 | next] → [10 | NULL]
```

Advantages: dynamic size and no fixed array capacity. **Disadvantage:** extra memory for links.

Important exam point: In an array stack, size is fixed when the array is created; a linked-list stack can grow/shrink dynamically subject to available memory.

3.2 APPLICATIONS OF STACK

1. **Reversing a list/string:** Push each item and then pop all items. Because of LIFO, the order is reversed.

Example: HELLO $\rightarrow$ push H,E,L,L,O $\rightarrow$ pop O,L,L,E,H $\rightarrow$ OLLEH.

2. **Polish notation:** Expressions can be written as infix, prefix or postfix.

Form	Example	Operator position
Infix	A + B	Between operands
Prefix	+ A B	Before operands
Postfix	A B +	After operands

Operator precedence: Parentheses $\rightarrow$ $\wedge \rightarrow$ *, /, % $\rightarrow$ +, -. Use associativity carefully, especially for exponentiation.

3. Infix to postfix conversion

Rules: scan left to right; operand goes directly to output; '(' is pushed; ')' causes operators to be popped until '('; for an operator, pop operators of higher precedence (and equal precedence where left-associative) before pushing the current operator; after scanning, pop remaining operators.

Worked example: $A + B * C \rightarrow ABC*+.$

Worked example: $(A + B) * C \rightarrow AB+C*.$

Worked example: $A * (B + C) \rightarrow ABC*+.$

4. Infix to prefix conversion

Common method: reverse the infix expression, interchange '(' and ')', convert the result to postfix, then reverse that postfix result.

Example: $A + B * C \rightarrow +A*BC.$

5. Postfix evaluation

Scan left to right. Operand $\rightarrow$ push. Operator $\rightarrow$ pop the top two operands, perform the operation as first-popped? Be careful: if the first popped value is b and second is a, calculate a operator b; push the result. The final stack value is the answer.

Example: $2\ 3 + 4 * \rightarrow 2+3=5 \rightarrow 5 \times 4=20.$

6. Prefix evaluation

Scan right to left. Operand $\rightarrow$ push. Operator $\rightarrow$ pop the first two operands, calculate first popped? For prefix, if a is the first popped and b is the second popped, calculate a operator b; push result. Final value is the answer.

Example: $+ 2\ 3 \rightarrow$ scan 3,2,+ $\rightarrow 2+3=5.$

7. **Recursion:** A function calling itself is recursion. It must have a base condition. Recursive calls are managed using the call stack.

Factorial example: $\text{factorial}(n) = n \times \text{factorial}(n-1)$, with $\text{factorial}(1)=1$. $\text{factorial}(4)=4 \times 3 \times 2 \times 1=24.$

3.3 INTRODUCTION TO QUEUE

Definition: A queue is a linear data structure in which insertion occurs at REAR and deletion occurs at FRONT. It follows FIFO (First In First Out).

Real-life example: People waiting in a ticket line. The person who enters first is normally served first.

FRONT $\rightarrow$ [10] [20] [30] [40] $\leftarrow$ REAR

Operations: Enqueue = insert at rear; Dequeue = delete from front; Peek = view front element; isEmpty = check empty; isFull = check full.

Linear array queue: FRONT and REAR are used to track the two ends. Exact initialization/empty conditions can vary by implementation, so follow the convention taught in your class.

Enqueue (common array idea): If full, report overflow. Otherwise increase REAR and store ITEM at QUEUE[REAR].

Dequeue (common array idea): If empty, report underflow. Otherwise read QUEUE[FRONT] and advance FRONT.

Queue Overflow: insertion attempted when the queue is full. **Queue Underflow:** deletion attempted when the queue is empty.

Linked-list queue: FRONT points to the first node and REAR points to the last node. Enqueue is performed at REAR and dequeue at FRONT.

3.4 TYPES OF QUEUE

1. Circular Queue

A circular queue treats the last array position as connected to the first. It reuses free spaces created at the beginning after deletions.

Typical full condition for a circular array queue: $(\text{REAR} + 1) \% \text{SIZE} == \text{FRONT}$. A common empty condition is $\text{FRONT} == -1$, depending on implementation.

Example for $\text{SIZE}=5$: if $\text{REAR}=4$, next position is $(4+1)\%5=0$.

Advantages: better use of array space and avoids the false-overflow problem of a simple linear queue.

2. Double Ended Queue (Deque)

A deque allows insertion and deletion at both ends.

Input-restricted deque: insertion is restricted to one end, but deletion can occur at both ends.

Output-restricted deque: deletion is restricted to one end, but insertion can occur at both ends.

Memory trick: Input restricted = input/insertion restricted. Output restricted = output/deletion restricted.

3. Priority Queue

A priority queue removes/serves elements according to priority rather than simply arrival order. If two items have the same priority, the implementation may use arrival order as a tie-breaker.

Ascending priority queue: smaller priority value is served first, under the convention where smaller value means higher priority. **Descending priority queue:** larger priority value is served first.

Example: priorities 1, 3, 2 → ascending service order 1,2,3; descending service order 3,2,1.

4. Applications of Queue

- Printer job scheduling
- CPU/process scheduling
- Keyboard/input buffering
- Call-center waiting systems
- Network packet buffering
- Breadth First Search (BFS) in graphs

HIGH-SCORING DIFFERENCES

Point	Stack	Queue
Principle	LIFO	FIFO
Insertion	TOP	REAR
Deletion	TOP	FRONT
Main ends used	One end	Two ends
Example	Stack of plates	Ticket line

Point	Linear Queue	Circular Queue
Shape	Linear	Circular/logically connected
Space reuse	Poor after front deletions	Efficient
Wrap-around	No	Yes
Typical full test	Depends on implementation	$(\text{REAR} + 1) \% \text{SIZE} == \text{FRONT}$

Overflow vs Underflow: Overflow = insertion into a full structure. Underflow = deletion from an empty structure.

CONVERSION & EVALUATION PRACTICE

Try each yourself before reading the answer.

1. Convert $A+B*C$ to postfix.

Answer: $ABC*+$

2. Convert $(A+B)*C$ to postfix.

Answer: $AB+C*$

3. Convert $A*(B+C)$ to postfix.

Answer: $ABC+*$

4. Convert $A+B-C$ to postfix.

Answer: $AB+C-$

5. Convert $A*(B+C)-D$ to postfix.

Answer: $ABC+*D-$

6. Convert $A+B*C$ to prefix.

Answer: $+A*BC$

7. Convert $(A+B)*C$ to prefix.

Answer: $*+ABC$

8. Evaluate $2\ 3\ +\ 4\ *$.

Answer: 20

9. Evaluate $8\ 2\ /\ 3\ +$.

Answer: 7

10. Evaluate $5\ 1\ 2\ +\ 4\ * +\ 3\ -$.

Answer: 14

11. Evaluate $+ 2\ * 3\ 4$.

Answer: 14

12. Evaluate $- * 2\ 3\ 4$.

Answer: 2

Extra challenge

13. Convert $(A+B)*(C-D)$ to postfix.

Answer: $AB+CD-*$.

14. Convert $A*(B-C)+D/E$ to postfix.

Answer: $ABC-*DE/+$.

15. Convert $(A+B)/(C-D)$ to prefix.

Answer: $/+AB-CD$.

C/C++ EXAM-FRIENDLY PROGRAMS

1. Stack using array (C++)

```
#include
using namespace std;

const int MAX = 5;
int st[MAX], top = -1;

void push(int x) {
    if (top == MAX - 1)
        cout << "Stack Overflow\n";
    else
        st[++top] = x;
}

void pop() {
    if (top == -1)
        cout << "Stack Underflow\n";
    else
        cout << "Deleted: " << st[top--] << "\n";
}

void peek() {
    if (top == -1)
        cout << "Stack Empty\n";
    else
        cout << "Top: " << st[top] << "\n";
}
```

2. Queue using array (basic linear queue)

```
#include
using namespace std;

const int MAX = 5;
int q[MAX], front = -1, rear = -1;

void enqueue(int x) {
    if (rear == MAX - 1)
        cout << "Queue Overflow\n";
    else {
        if (front == -1) front = 0;
        q[++rear] = x;
    }
}

void dequeue() {
    if (front == -1 || front > rear)
        cout << "Queue Underflow\n";
    else
        cout << "Deleted: " << q[front++] << "\n";
}
```

For exams, write the exact version/initialization convention taught by your teacher. The logic is what matters: stack uses TOP; queue uses FRONT and REAR.

READY-TO-WRITE EXAM ANSWERS

2 marks — Define stack.

A stack is a linear data structure in which insertion and deletion take place only at one end called TOP. It follows LIFO (Last In First Out).

2 marks — Define queue.

A queue is a linear data structure in which insertion is performed at REAR and deletion at FRONT. It follows FIFO (First In First Out).

4 marks — Explain Push and Pop.

Push inserts an element at TOP after checking overflow. Pop removes the element from TOP after checking underflow. Both operations take $O(1)$ time in a standard array implementation.

4 marks — Explain circular queue.

A circular queue connects the last position logically to the first position. It allows reuse of empty positions created by deletions. A common full condition is $(\text{REAR}+1)\% \text{SIZE} == \text{FRONT}$.

4 marks — Explain deque types.

A deque permits insertion and deletion at both ends. Input-restricted deque restricts insertion to one end while deletion is allowed at both. Output-restricted deque restricts deletion to one end while insertion is allowed at both.

6 marks — Explain applications of stack.

Stack is used for reversing data, expression conversion among infix/prefix/postfix, postfix/prefix evaluation, recursion/call-stack management, and other backtracking/processing tasks.

6 marks — Explain priority queue.

A priority queue is a queue in which service/removal depends on priority. In an ascending priority queue, the smallest priority value may be served first; in a descending priority queue, the largest may be served first. It is useful in scheduling and other priority-based processing.

MCQs — SELF TEST

1. Stack follows:

- A) FIFO B) LIFO C) Random D) Priority

Answer: B

2. In a stack, insertion is called:

- A) Enqueue B) Dequeue C) Push D) Peek

Answer: C

3. In a queue, insertion occurs at:

- A) FRONT B) TOP C) REAR D) Middle

Answer: C

4. Deletion from a queue is called:

- A) Pop B) Dequeue C) Push D) Insert

Answer: B

5. Trying to pop an empty stack causes:

- A) Overflow B) Underflow C) Collision D) Wrap

Answer: B

6. Which is a stack application?

- A) Expression conversion B) Printer queue only C) FIFO buffering only D) None

Answer: A

7. BFS generally uses:

- A) Stack B) Queue C) Heap only D) Array only

Answer: B

8. A deque permits:

- A) Only insertion at front B) Only deletion at rear C) Insertion and deletion at both ends D) No deletion

Answer: C

9. Circular queue mainly helps to:

- A) Sort data B) Reuse queue space C) Reverse strings only D) Evaluate prefix only

Answer: B

10. Postfix expression for $A+B*C$ is:

- A) $+A*BC$ B) $ABC*+$ C) $AB+C*$ D) $*+ABC$

Answer: B

11. Prefix expression for $A+B*C$ is:

- A) $ABC*+$ B) $+A*BC$ C) $AB+C*$ D) $*ABC+$

Answer: B

12. Which operation only reads the top element?

- A) Pop B) Push C) Peek D) Enqueue

Answer: C

VIVA / ORAL QUESTIONS

1. What is LIFO?
2. What is FIFO?
3. What is TOP in a stack?
4. What is the difference between push and pop?
5. What is stack overflow?
6. What is stack underflow?
7. What is the purpose of peek?
8. What is an ADT?
9. What is an infix expression?
10. What is a postfix expression?
11. What is a prefix expression?
12. Why is a stack used in expression conversion?
13. In which direction is a prefix expression evaluated?
14. In which direction is a postfix expression evaluated?
15. What is recursion?
16. What is the call stack?
17. What are FRONT and REAR in a queue?
18. What is enqueue?
19. What is dequeue?
20. What is a circular queue?
21. What is the full condition commonly used for a circular queue?
22. What is a deque?
23. What is an input-restricted deque?
24. What is an output-restricted deque?
25. What is a priority queue?
26. Give two applications of queue.
27. Give two applications of stack.

30-MARK MODEL QUESTION PAPER

Attempt without notes. Suggested time: 45–60 minutes.

Q1. Answer any five: ($5 \times 2 = 10$)

- Define stack and state its principle.
- Define queue and state its principle.
- Define overflow and underflow.
- Differentiate prefix and postfix.
- What is a deque?
- What is a circular queue?

Q2. Answer any two: ($2 \times 4 = 8$)

- Explain push and pop with algorithms.
- Explain enqueue and dequeue with suitable diagrams.
- Explain input-restricted and output-restricted deque.

Q3. Solve any two: ($2 \times 3 = 6$)

- Convert $A+B*C-D$ to postfix.
- Convert $(A+B)*C$ to prefix.
- Evaluate $2\ 3 + 4 * 5 -$.

Q4. Answer any one: ($1 \times 6 = 6$)

- Explain applications of stack including expression conversion/evaluation and recursion.
- Explain circular queue and priority queue with examples and applications.

MODEL PAPER ANSWER KEY

Q3(a): $ABC*+D-$. Q3(b): $*+ABC$. Q3(c): $2+3=5$, $5 \times 4=20$, $20-5=15$.

LAST-DAY REVISION PLAN

- 30 min: Stack definition, LIFO, push/pop, overflow/underflow, array and linked-list representation.
- 45 min: Infix → postfix and infix → prefix rules. Solve at least 5 conversions.
- 30 min: Postfix and prefix evaluation. Solve at least 5 expressions.
- 20 min: Recursion and call stack.
- 30 min: Queue, FIFO, enqueue/dequeue, overflow/underflow, array/linked-list representation.
- 25 min: Circular queue, deque and its two types, priority queue.
- 20 min: Stack vs queue and linear vs circular queue differences.
- 30–45 min: Attempt the 30-mark model paper without notes.
- Final 10 min: Revise formulas/rules: LIFO, FIFO, $(\text{REAR}+1)\% \text{SIZE}$, precedence and conversion directions.

30/30 checklist

- I can define every term in 1–2 precise sentences.
- I can draw stack, queue, circular queue and deque diagrams.
- I can write push/pop/enqueue/dequeue algorithms.
- I can identify overflow and underflow.
- I can convert infix to postfix and prefix.
- I can evaluate postfix left-to-right and prefix right-to-left.
- I can explain recursion and the call stack.
- I can distinguish input-restricted and output-restricted deque.
- I can explain priority queue and its types.
- I can write differences in table form.
- I can solve a 30-mark paper within the allotted time.